

Biblioteca API - Ejercicio Parte 3

En **DOSW Company**, el equipo de ingeniería celebra, ya que después de semanas de trabajo, el primer prototipo del sistema de gestión de la biblioteca ha sido un éxito!! Cada uno de ustedes logró construir un MVP funcional, sólido y alineado con las necesidades iniciales del cliente, utilizando persistencia relacional con **JPA** y **Hibernate**.

Sin embargo, como suele suceder en el mundo real... los requerimientos no se detienen.

Durante una reciente reunión, el cliente plantea un nuevo escenario: **“Nuestro catálogo está creciendo... pero no todos los libros son iguales.”**

Algunos nuevos cambios ahora incluyen:

- Datos extendidos:
 - **Libro:** Ahora es necesario almacenar dentro de cada libro adicional a lo que ya tenía, las categorías a las que pertenece, el tipo de publicación (Revista, ebook, cartilla, etc), la fecha de publicación, el código isbn, los datos de la metadata (páginas, idioma, Empresa que lo público), La disponibilidad (status, número total de copias, número de copias disponibles, número de copias prestadas) y la fecha en la que fue agregado al catalogo.
 - **Usuario:** Ahora es necesario almacenar dentro de cada usuario adicional a lo que ya tenía, su correo electrónico, el tipo de membresía que tiene (VIP, Platinum, standard), y la fecha en la que fue agregado como usuario a la biblioteca.
 - **Préstamo:** Ahora es necesario almacenar dentro de cada préstamo adicional a lo que ya tenía, el historial sobre el mismo préstamo (status, y en que fecha fue ejecutado)

El modelo actual, basado en tablas rígidas, comienza a mostrar sus límites. Además, el cliente agrega:

“Estamos pensando en escalar esto. Queremos que el sistema esté preparado para crecer sin fricciones.”

El equipo arquitectónico propone una solución: Incorporar una base de datos NoSQL usando **MongoDB Atlas**, que permita manejar estructuras flexibles y facilite la escalabilidad.

Pero hay una condición crítica: **El sistema actual no puede desaparecer.** La persistencia relacional debe seguir funcionando mientras se introduce la nueva solución.

Ahora el sistema deberá vivir en **dos mundos al mismo tiempo**.

Cuando creías que el reto terminaba ahí, el cliente agrega un nuevo requerimiento:

“Ya no queremos ver esto corriendo solo en local. Necesitamos probarlo con usuarios reales.”



El sistema debe ser desplegado en la nube utilizando **Microsoft Azure**, permitiendo acceso a múltiples usuarios de prueba.

Y no solo eso... El cliente, con un equipo cada vez más maduro, exige prácticas modernas de desarrollo:

“Queremos que todo el proceso sea automático. Cada cambio debe construirse, probarse y desplegarse sin intervención manual.”

Para esto, deberán implementar un pipeline de **CI/CD** usando **GitHub Actions**, que orquesta:

- Construcción del proyecto
- Ejecución de pruebas
- Análisis estático de código
- Despliegue en la nube en el entorno Productivo

Tu misión

Ahora eres parte del equipo encargado de esta evolución.

Debes diseñar e implementar una solución que:

- Permita que el sistema soporte **persistencia dual** (relacional + NoSQL)
- Mantenga la compatibilidad con el MVP existente
- Automatice el ciclo de vida de la aplicación mediante CI/CD
- Despliegue la solución en la nube (Azure)
- Garantice calidad mediante **tests funcionales clave del dominio**

Hora del juego

DOSW Company no solo quiere una solución... Quiere identificar el nivel actual de su equipo de ingenieros antes de la prueba Técnica que se va a desarrollar en Semana 11.

Por eso, han convertido el reto en una arena competitiva donde:

- Cada decisión técnica cuenta
- El tiempo importa
- Y ahora...puedes sabotear a otros ingenieros

Tienes que ser muy estratégico con tus decisiones, recuerda que por cada reto completado correctamente obtienes una recompensa a elegir: Un Poder o un Saboteo. **(No puedes usar un saboteo sobre la misma persona 2 veces seguidas y tienes que utilizar el poder o el saboteo inmediatamente o sino lo perderás)**

Sistema de Puntuaciones:

Completar el reto +100 puntos

Pipeline exitoso +50 puntos

Deploy funcional +100 puntos

Resolver sabotaje +70 puntos

Uso estratégico del poder +30 puntos

Fallo crítico en la presentación -50 puntos

Recompensas académicas

Resultado en el hackatón	Beneficio
 1er lugar	+1.0 sobre la nota del parcial
 2do lugar	+0.7 sobre la nota
 3er lugar	+0.5 sobre la nota
Top 5	+0.3 sobre la nota
Participación completa	+0.2

Nota importante

 La suma de puntos **puede mejorar significativamente tu nota**, pero no reemplaza el parcial.

El hackatón se desarrolla **principalmente en clase donde** Se evaluará:

- Participación activa
- Toma de decisiones
- Resolución de problemas en tiempo real
- ⚠ Sabemos que esto simula un entorno real. Por eso:
 - Si no alcanzas a terminar:
 - Puedes completarlo en casa
 - Pero:
 - Lo avanzado en clase tiene mayor peso evaluativo

RECUERDA: Tienes que subir tu avance antes de finalizar la clase, agregando de manera adicional a la entrega en qué puesto quedaste, y cuantos retos lograste completar en clase.

HACKATHON CORTE 2 - Library wars

Reto #1 - El cambio del esquema:

- Para este primer reto debes generar tu modelo no relacional en base a los cambios que el enunciado te plantea - recuerda tienes que postular como seria tu meta modelo conceptual, que relaciones tienen entre los objetos, que documentos se generan y si son embebidos o referenciados.

Reto #2 - La conexión con Mongo:

1. Crea un cluster gratis con Mongo Atlas ingresando en el siguiente link <https://www.mongodb.com/cloud/atlas/register>
2. Llena todos los detalles e inicia a crear tu primer cluster:
 - a. Selecciona la opción M0 que es para entorno educativo y no tiene costo
 - b. En quick setup selecciona Automate security setup
 - c. No cambies el nombre del cluster creado
 - d. Selecciona como proveedor AWS
 - e. La región va a ser la recomendada según tu selección
 - f. No agregues información en la sección que maneja el Tag
 - g. Dale click a Create Deployment
3. El siguiente paso es establecer la conexión con la base de datos MongoDB Atlas. Para ello, necesitamos obtener la cadena de conexión del clúster, que se encuentra en la interfaz de usuario de Atlas.

Connect to cluster0

Set up connection security Choose a connection method Connect

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver Version

Java 4.3 or later

2. Install your driver

[View MongoDB Java Driver Installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☐ View full code sample

mongodb+srv://theuser:<db_password>@cluster0.k5dqp.mongodb.net/?retryWrites=true&w=majority&appName=cluster0

Replace <db_password> with the password for the theuser database user. Ensure any option params are URL encoded.

RESOURCES

[Get started with the Java Driver](#) [Java Starter Sample App](#)

[Access your Database Users](#) [Troubleshoot Connections](#)

Go Back Done

4. Copia los datos de conexión y guárdalos en un archivo de texto
5. Descarga Mongo Compass y prueba con los datos previamente guardados la conexión con tu base de datos.

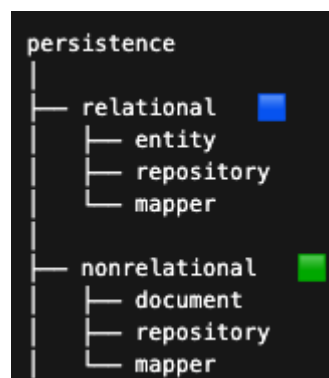
Reto #3 - La Implementación con Mongo:

1. Con spring initializer selecciona las configuraciones actuales de tu proyecto y selecciona como dependencia Spring Data MongoDB y descarga el proyecto generado.
2. Una vez descargado copia la dependencia que se genera en ese pom.xml y agrégalo a tu pom del proyecto de la biblioteca (recuerda utilizar maven para el ciclo de vida de tu app y que pueda instalar la dependencia correctamente).
3. Una vez que obtenga la conexión desde la interfaz de usuario de MongoDB Atlas, pegue la cadena generada en el archivo application.yaml o application.properties (depende de cual estes manejando)

```
1 spring.data.mongodb.uri=mongodb+srv://<username>:<pwd>@<cluster>.mongodb.net/mygrocerylist
2 spring.data.mongodb.database=mygrocerylist
```

```
spring:
  data:
    mongodb:
      uri: mongodb+srv://<username>:<pwd>@<cluster>.mongodb.net/mygrocerylist
      database: mygrocerylist
```

4. No es necesario escribir código relacionado con la conexión en ningún otro archivo. Spring Boot se encarga de la conexión a la base de datos - Si te falla recuerda agregar el `@EnableMongoRepositories` en tu main.
5. Divide la capa de persistencia en 2 : relational y non Relational. quedando de la siguiente manera:



6. Recuerda que así como modificaste tus carpetas, así mismo tienes que modificar las de tus pruebas.

7. A partir de lo generado en el reto # 1 define las entidades de persistencia - Documentos que estableciste creando una nueva carpeta que se llame document en la capa de persistencia - Ahí crearas cada uno de los documentos de la solución.
8. Crea los repositorios necesarios para la implementación utilizando como extensión Mongo Repository, con ellos puedes gestionar cada uno de los CRUD que necesitas por ítem.

```
public interface ItemRepository extends MongoRepository<GroceryItem, String> {

    @Query("{name:'?0'}")
    GroceryItem findItemByName(String name);

    @Query(value="{category:'?0'}", fields="{ 'name' : 1, 'quantity' : 1}")
    List<GroceryItem> findAll(String category);

    public long count();
}
```

Reto #4 - La decisión de relacional vs No Relacional:

1. Crea una interfaz general para los repositorios este es un ejemplo:

```
public interface LoanRepository {

    Loan save(Loan loan);

    Optional<Loan> findById(String id);

    List<Loan> findAll();

    void delete(String id);
}
```

2. A partir de este cambio es necesario dentro de los servicios cambiar la inyección y que el servicio quede de este estilo:

```
@Service
public class LoanService {

    private final LoanRepository loanRepository;

    public LoanService(LoanRepository loanRepository) {
        this.loanRepository = loanRepository;
    }

    public Loan createLoan(Loan loan) {
        return loanRepository.save(loan);
    }

    public List<Loan> getAll() {
        return loanRepository.findAll();
    }

    public void delete(String id) {
        loanRepository.delete(id);
    }
}
```

3. Creen Los repositorios concretos por cada tipo de implementación relacional, como se puede ver en el siguiente ejemplo:

```
@Repository
@Profile("relational")
public class LoanRepositoryJpaImpl implements LoanRepository {

    private final JpaLoanRepository repository;
    private final LoanEntityMapper mapper;

    public LoanRepositoryJpaImpl(JpaLoanRepository repository,
                                LoanEntityMapper mapper) {
        this.repository = repository;
        this.mapper = mapper;
    }

    @Override
    public Loan save(Loan loan) {
        return mapper.toDomain(
            repository.save(mapper.toEntity(loan))
        );
    }
}
```

```
@Override
public Loan save(Loan loan) {
    return mapper.toDomain(
        repository.save(mapper.toEntity(loan))
    );
}

@Override
public List<Loan> findAll() {
    return repository.findAll()
        .stream()
        .map(mapper::toDomain)
        .toList();
}

@Override
public void delete(String id) {
    repository.deleteById(Long.valueOf(id));
}
}
```

4. Creen Los repositorios concretos por cada tipo de implementación no relacional como se puede ver en el siguiente ejemplo:

```
@Repository
@Profile("mongo")
public class LoanRepositoryMongoImpl implements LoanRepository {

    private final MongoLoanRepository repository;
    private final LoanDocumentMapper mapper;

    public LoanRepositoryMongoImpl(MongoLoanRepository repository,
                                    LoanDocumentMapper mapper) {
        this.repository = repository;
        this.mapper = mapper;
    }

    @Override
    public Loan save(Loan loan) {
        return mapper.toDomain(
            repository.save(mapper.toDocument(loan))
        );
    }
}
```

```
@Override
public Loan save(Loan loan) {
    return mapper.toDomain(
        repository.save(mapper.toDocument(loan))
    );
}

@Override
public List<Loan> findAll() {
    return repository.findAll()
        .stream()
        .map(mapper::toDomain)
        .toList();
}

@Override
public void delete(String id) {
    repository.deleteById(id);
}
}
```

5. Utilice el manejo de streams para la utilización de sus operaciones dentro de los repositorios.
6. En su configuración application.yaml o application.properties agregue dentro de las configuraciones los perfiles - esto les va a permitir que spring detecte por cada uno si utiliza mongo o relational con JPA y lo inyecta automáticamente en el servicio.

```
spring:
  profiles:
    active: mongo
```

```
SPRING_PROFILES_ACTIVE=relational
```

7. Deja el profile activo como mongo, una vez activado tienes que reiniciar tu aplicación

Reto # 5 - Pruebas funcionales:

1. Una vez implementado ejecuta tu código, recuerda que como tienes implementado seguridad en tu API todo el tema de credenciales debe estar también reflejado en tu BD de mongo.
2. Desde tu swagger UI, ejecuta cada una de las pruebas funcionales, simulando un entorno real de la biblioteca.
3. Graba un video y adjunta las pruebas funcionales con MongoDB como un anexo a tu README en el proyecto.

Reto # 6 - CI/CD Implementado inicialmente:

1. Configurar en github actions un workflow que contendrá 4 jobs, el primer job se llamará build, el segundo test, el tercero analysis y el cuarto deploy, además, este workflow se disparará (events/trigger) on: pull_request, para esto puede usar como base [este tutorial](#) o cualquier otro que considere. Deberás hacer steps or actions sobre cada job para que se pueda obtener los siguiente:

- a. **build**: únicamente realizar hasta la fase *compile* de *maven*, es prerequisite de test.
 - b. **test**: realizar la fase de *verify* y responder ¿se puede lograr que se ejecute sin necesidad de compilar el proyecto?, es dependiente (**needs**) de build para ser ejecutada, y a su vez es prerequisite para hacer el análisis estático.
 - c. **analysis**: Este job estará enfocado en análisis del código, por ejemplo:
 - Validación de calidad (linters o plugins de Maven)
 - Revisión estática de código
 - Reporte de cobertura (si lo implementas)
 - d. **deploy**: por ahora deberá imprimir en consola "En construcción ...", necesita (**needs**) que se haya ejecutado **test** antes de iniciar.
2. Agregar los siguientes tests
 - Dado que tengo 1 reserva registrada, Cuando lo consulto a nivel de servicio, entonces la consulta será exitosa validando el campo id.
 - Dado que no hay ninguna reserva registrada, Cuando la consultó a nivel de servicio, Entonces la consulta no retorna ningún resultado.
 - Dado que no hay ninguna reserva registrada, Cuando lo creo a nivel de servicio, entonces la creación será exitosa.
 - Dado que tengo 1 reserva registrada, Cuando la elimino a nivel de servicio, entonces la eliminación será exitosa.
 - Dado que tengo 1 reserva registrada, Cuando la elimino y consulto a nivel de servicio, entonces el resultado de la consulta no retorna ningún resultado.
 3. Verifica que la ejecución del **workflow** es exitosa, si no lo fuera, modifícalo hasta que ocurra

Reto # 7:

1. En Azure crea un servicio de App Service con recursos que facturen 0 dólares ([TUTORIAL MINUTO 4:34](#))
2. Configura el job deploy que creaste en el reto #6 y usando el *action* `azure/webapps-deploy@v2` despliega el *jar* generado a tu servicio de App Service
3. Verifica qué el endpoint de la aplicación generado en App Service
4. En este punto la aplicación no debería funcionar, ¿Donde puedes ver el mensaje de error de la aplicación o logs?, (probablemente está fallando debido a que el puerto usado para despliegue no es el esperado (puerto 80), modifícalo en el `application.properties`)
5. Para utilizar la base de datos, configura los datos de conexión como una o varias variables de entorno tanto en App Service como en el archivo de configuraciones de tu proyecto.
6. Agrega **SPRING_DATA_MONGODB_URI** a las configuraciones dentro de las variables de entorno, guarda los cambios y reinicia la aplicación.
7. Prueba nuevamente la aplicación, ya debería estar funcionando!
8. Realiza las pruebas funcionales apuntando a la URL de tu despliegue, graba un video y agrégalo como anexo de **CI/CD** a tu **README**.

Reto #8:

1. Realiza un documento de high level design de arquitectura de back end, donde plantees la arquitectura de tu sistema de Librería.
2. Adjunta el documento en tu proyecto.

Mensaje final

Mucha suerte en este reto. Este es un espacio para demostrar lo que sabes hacer **como ingeniero de manera individual**: cómo piensas, cómo diseñas y cómo resuelves problemas cuando las cosas no salen como esperas.

Te vas a encontrar con errores, bloqueos y decisiones difíciles... y eso es justamente lo que buscamos evaluar.

No se trata de hacerlo perfecto, sino de **entender, iterar y avanzar**.

 Recuerda: en la vida real, nadie te va a dar el camino completo — lo importante es cómo lo construyes.

 Confía en tu criterio, mantente enfocado y no te detengas ante el primer fallo.

 ¡Mucho éxito! esto ya es nivel producción!